



# Architettura degli elaboratori

## Lezione 9 - teoria

*Prof.ssa Valentina Ciriani*

Università degli Studi di Milano

[www.di.unimi.it/ciriani](http://www.di.unimi.it/ciriani)

1



## Contenuto

- Progetto CPU a singolo ciclo [PH 4.1-4.4]
  - 1. Fase di fetch
  - 2. Istruzioni di tipo R
  - 3. Istruzioni di tipo I
    - 1. lettura in memoria
    - 2. scrittura in memoria
    - 3. salto condizionato
  - 4. Istruzioni di tipo J salto incondizionato

-MKM = M. Morris Mano, C.R. Kime, T. Martin, Reti logiche, Pearson

-PH = D.A. Patterson, J.L. Hennessy, Struttura e Progetto dei Calcolatori, Zanichelli

2

2



## Implementazione HW

- L'architettura delle istruzioni di un processore
  - non corrisponde ad una particolare implementazione hardware
  - può avere implementazioni diverse
- Noi vedremo una implementazione hardware di un processore semplificato che esegue un **sottoinsieme** delle istruzioni MIPS:
  - istruzioni logico-matematiche (R):  
add, sub, and, or, slt
  - istruzioni di lettura e scrittura in memoria (I):  
lw, sw
  - istruzione di salto condizionato (I):  
beq
  - istruzione di salto incondizionato (J):  
j

3

3

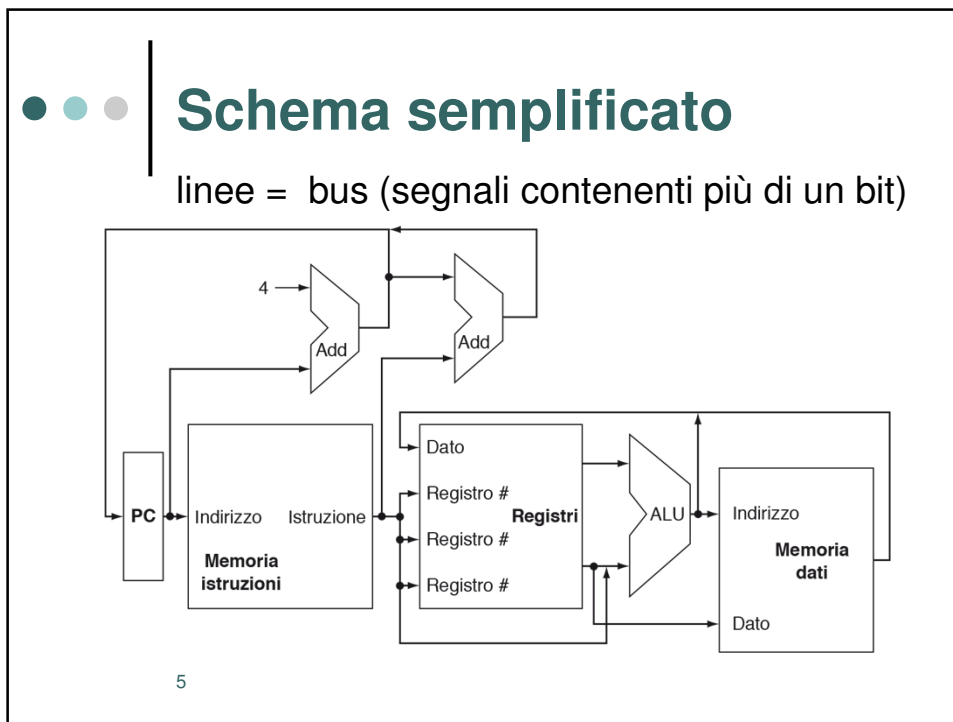


## CPU

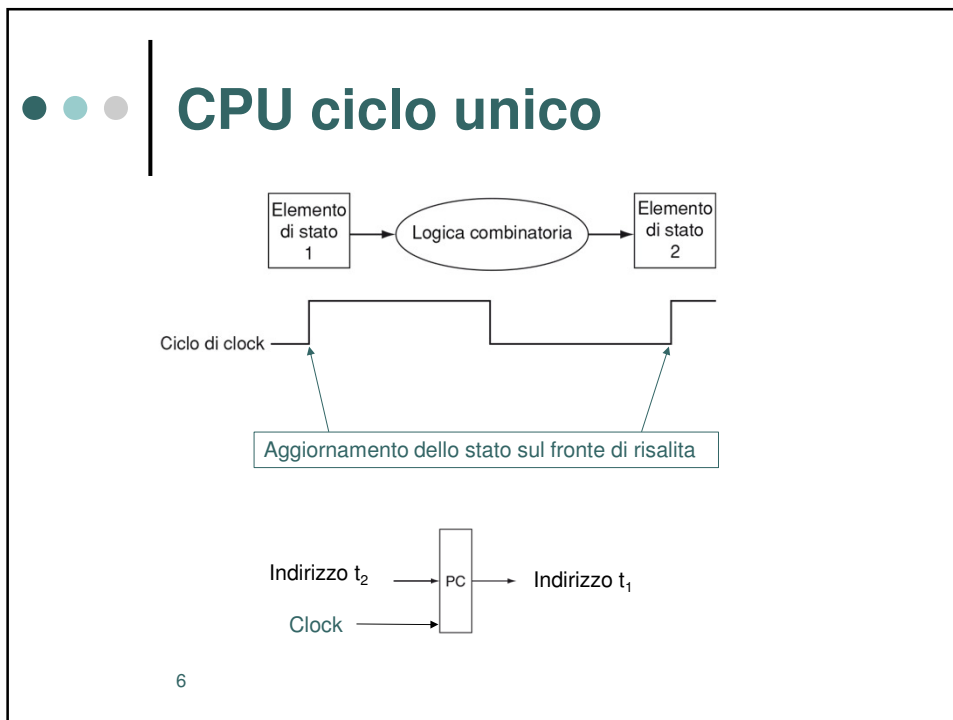
- La CPU è costituita da:
  1. l'**unità di controllo** (control unit)
  2. l'**unità di elaborazione** (data path)
- 1. L'**unità di controllo** riceve in input un'istruzione, la interpreta e genera **segnali di controllo** per istruire l'unità di elaborazione
- 2. L'**unità di elaborazione** esegue le istruzioni utilizzando elementi di stato ed elementi combinatori tra loro connessi

4

4

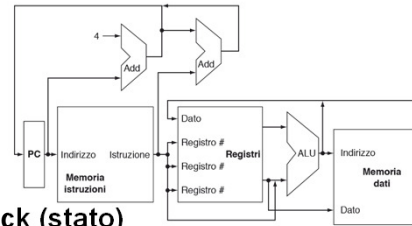


5



6

## Schema semplificato



- **Elementi che variano con il clock (stato)**
  - PC (program counter)
  - Registri generali
  - Memoria dati
    - scrittura di un dato sulla base dell'indirizzo
- **Elementi combinatori (indipendenti dal clock):**
  - ALU (unità aritmetico logica)
    - esegue le operazioni aritmetiche e logiche,
    - utilizzata *da tutte* le istruzioni *tranne* la jump
  - Sommatori (Add)
  - Memoria istruzioni
    - lettura di una istruzione dalla memoria centrale
  - Memoria dati
    - lettura di un dato sulla base dell'indirizzo

7

7

## Metodo di temporizzazione

- Gli **elementi di stato** hanno in ingresso almeno 2 valori:
  - il valore da scrivere nell'elemento
  - Il clock (determina quando lo stato può essere aggiornato)
- Il **metodo di temporizzazione** (quando lo stato è aggiornato):
  - temporizzazione **sensibile ai fronti**, lo stato viene aggiornato in corrispondenza di un fronte del segnale del clock (fronte di salita o discesa)
- Assumiamo che:
  - lo stato venga aggiornato sul **fronte di salita**
  - **ogni istruzione dura 1 ciclo di clock**

8

8

# Fetch

9

9

## Fase di fetch

**Fetch:** caricamento di un'istruzione

1. **PC**

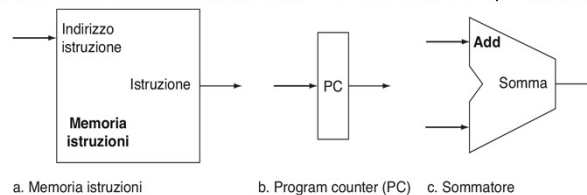
1. contiene l'indirizzo dell'istruzione corrente
2. deve essere aggiornato al valore  $PC+4$  (istruzione successiva) se non ci sono salti

2. **Sommatore:**

1. circuito combinatorio che calcola la somma tra i due ingressi a 32 bit
2. il risultato è l'indirizzo della prossima istruzione

3. **Memoria istruzioni:**

1. è un circuito combinatorio
2. legge l'istruzione dalla memoria centrale all'indirizzo specificato



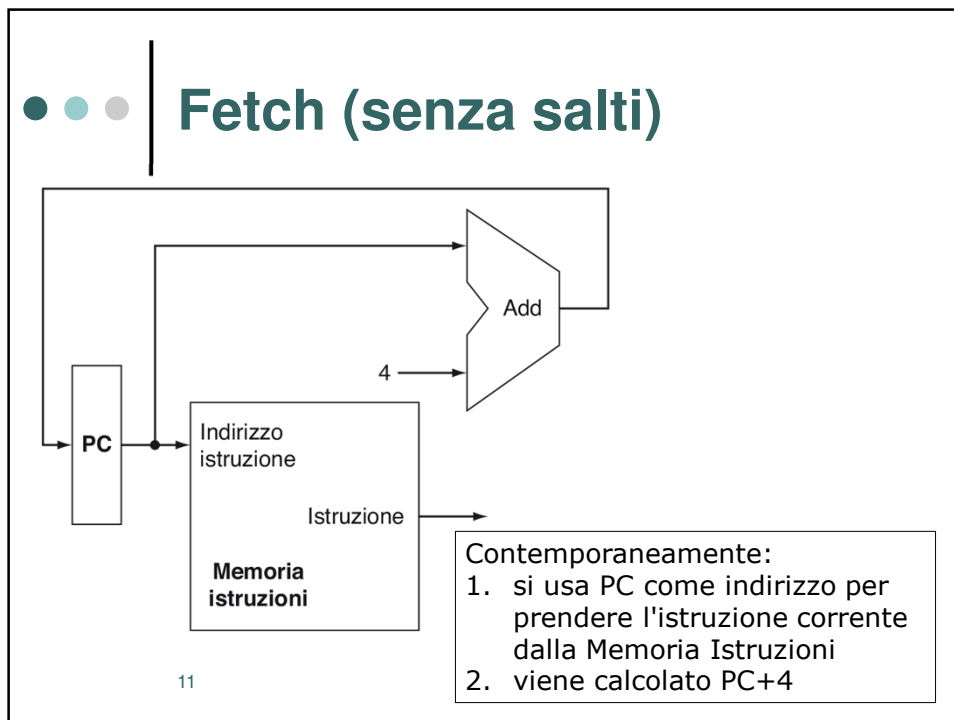
10

a. Memoria istruzioni

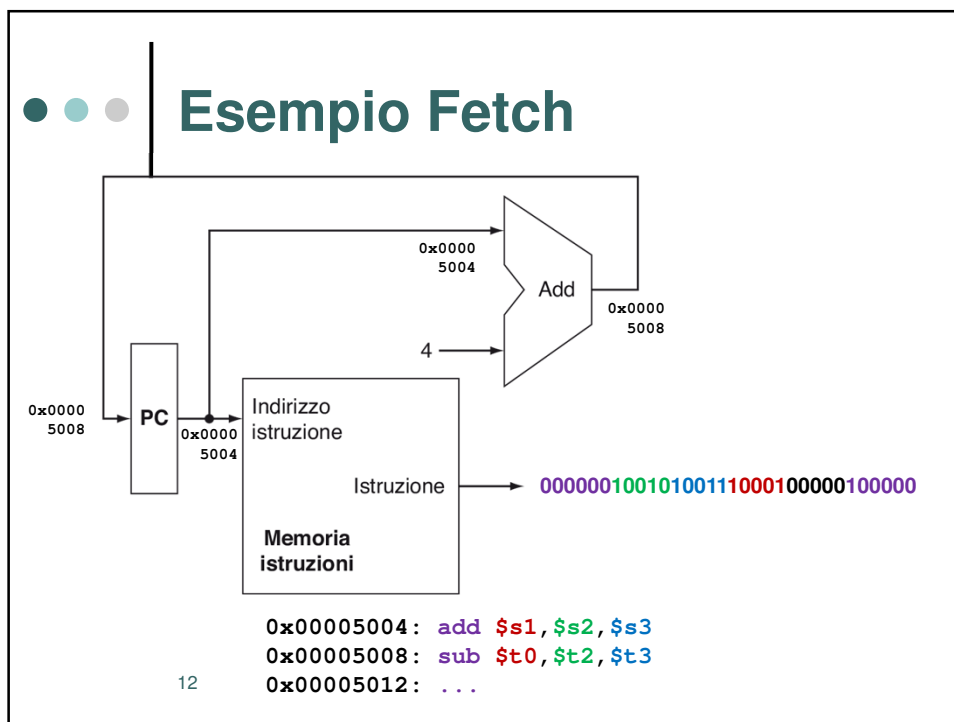
b. Program counter (PC)

c. Sommatore

10



11



12

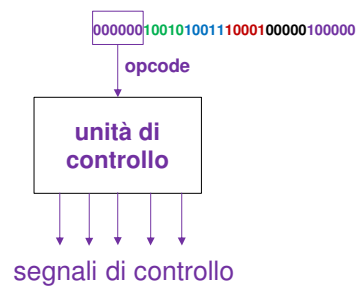
# Decodifica

13

13

## Fase di decodifica

1. lettura dell'istruzione
2. generazione dei segnali di controllo opportuni
3. lettura dei registri (tranne jump)



14

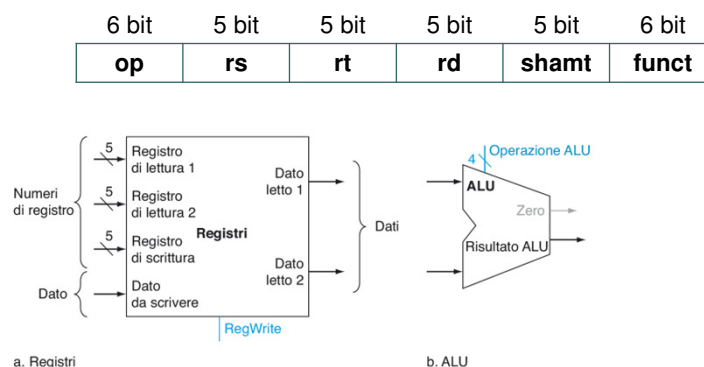
14

## Esecuzione delle istruzioni di formato R

15

15

## Sequenza azioni



1. Lettura dei registri **rs** e **rt**
2. Operazione **funct** nell'ALU
3. Scrittura del risultato nel registro **rd**

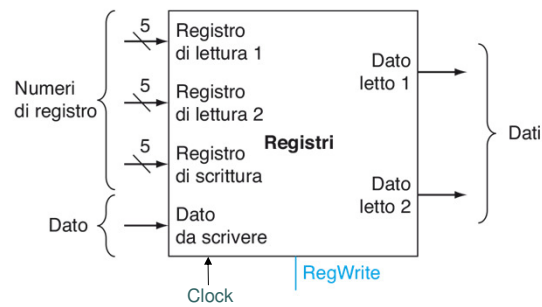
16

16





## Registri



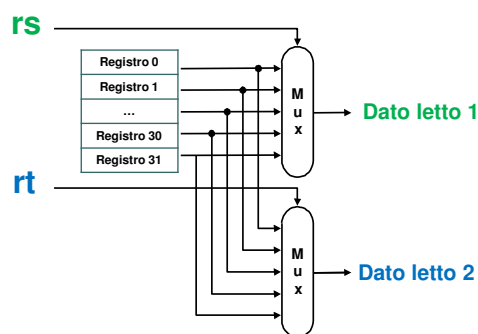
1. **rs** e **rt** sono i registri da leggere
2. **rd** è il registro dove scrivere il Dato
3. **RegWrite** abilita la scrittura (la scrittura deve essere indicata esplicitamente, la lettura no)

17

17



## Lettura Registri

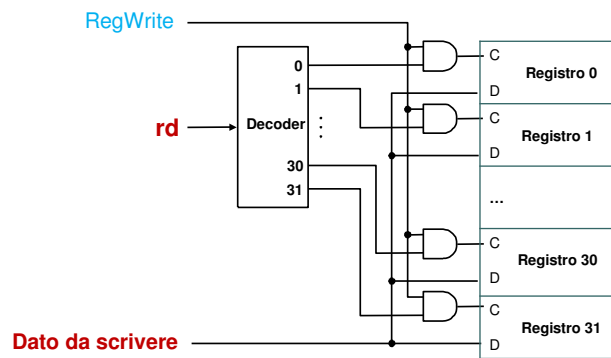


18

18



## Scrittura Registri

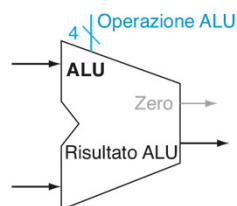


19

19



## ALU

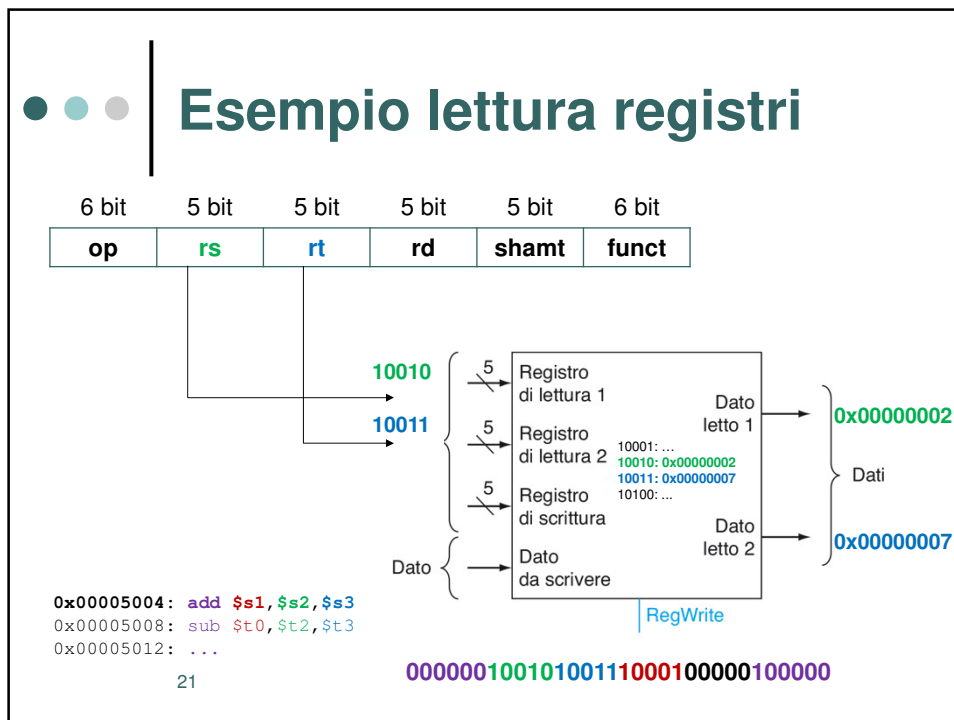


### Unità aritmetico-logica

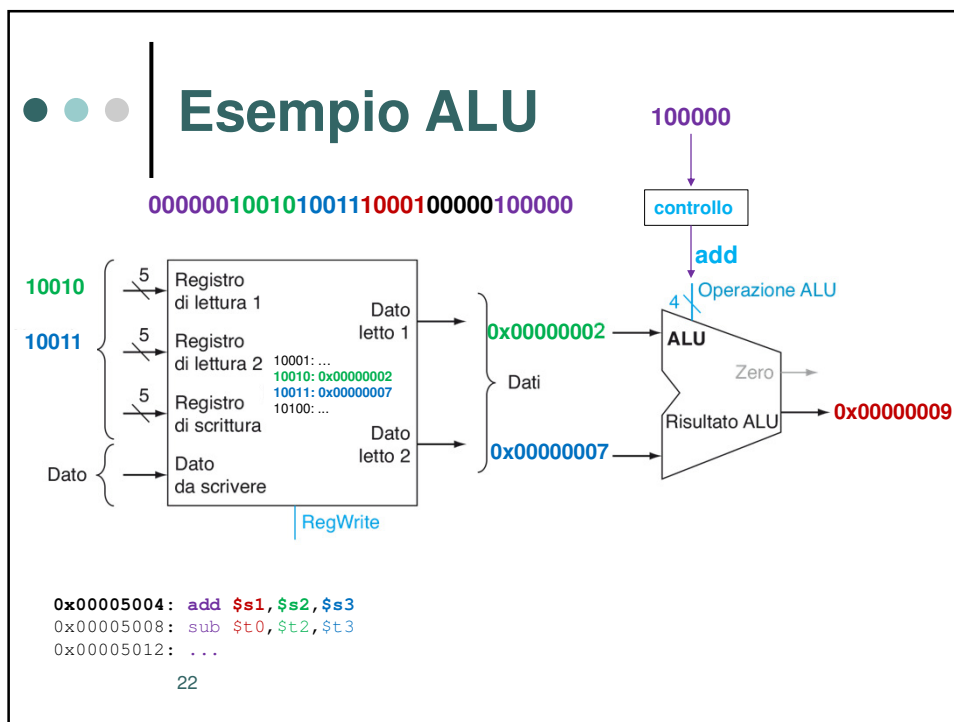
- In **ingresso** ha
  - due operandi di 32 bit
  - OperazioneALU (per specificare l'operazione da fare)
- Da in **output**
  - un risultato su 32 bit,
  - un bit (Zero) che vale 1 se il risultato dell'operazione è 0 (si usa per beq)

20

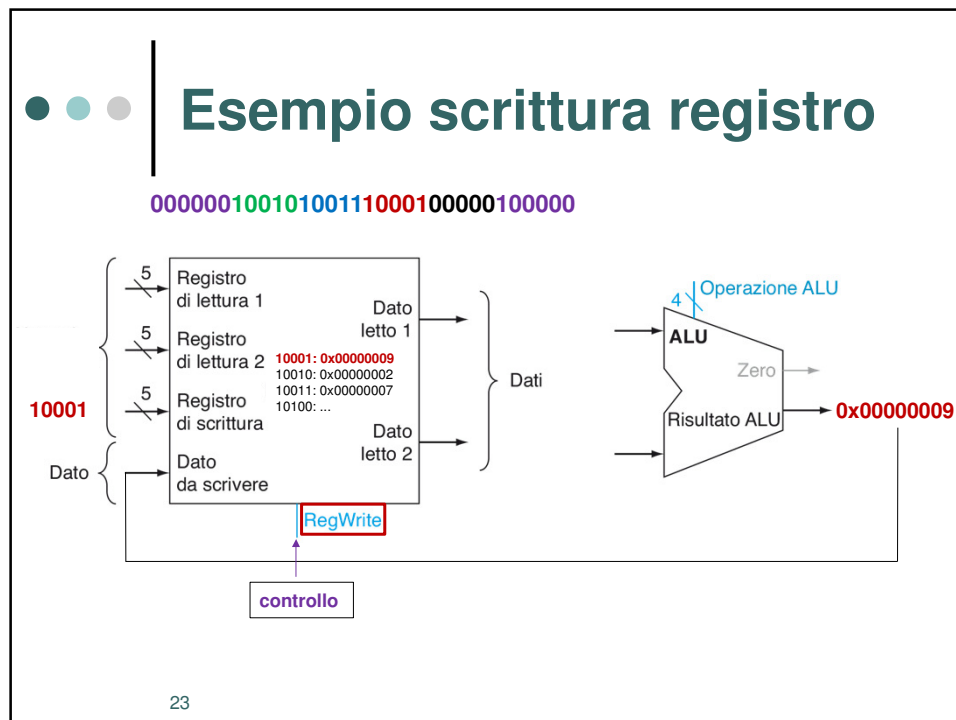
20



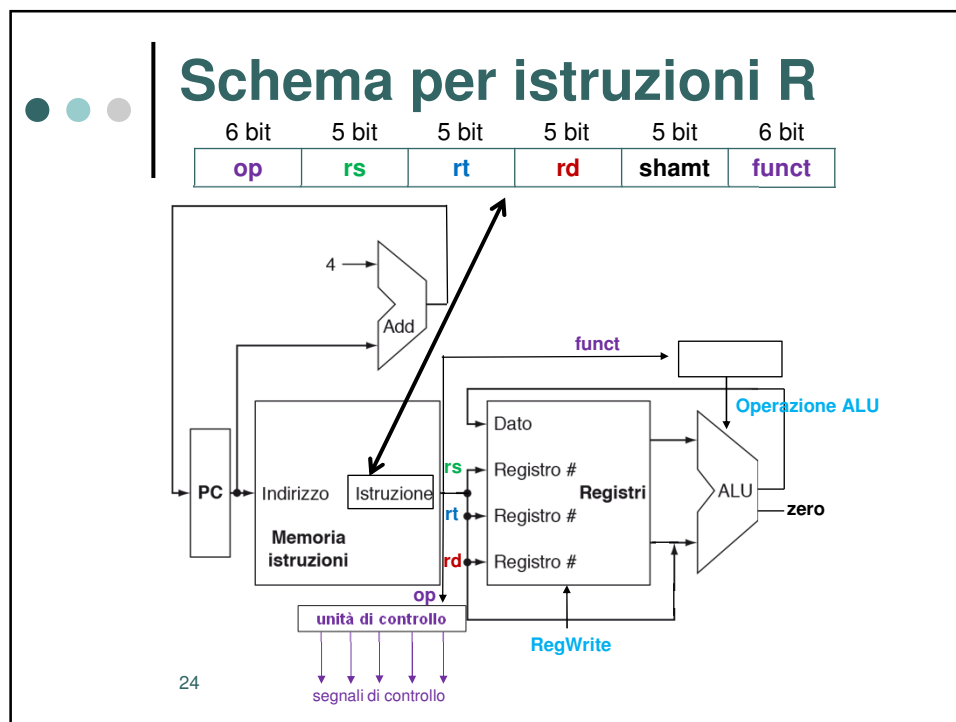
21



22



23



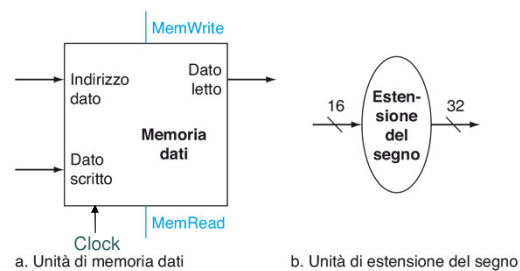
24

## Esecuzione delle istruzioni di formato I (lettura/scrittura in memoria)

25

25

## Unità aggiuntive utilizzate



### a) Unità di memoria dati:

- un circuito combinatorio usato per accedere in memoria centrale
- ha in ingresso 2 segnali di controllo: MemRead MemWrite che Indicano quale operazione deve essere effettuata

### b) L'unità di estensione del segno:

- inserisce nei 16 bit più significativi il bit di segno della costante (o tutti 1 o tutti 0)

26

26



## Istruzioni lw e sw

| 6 bit | 5 bit | 5 bit | 16 bit             |
|-------|-------|-------|--------------------|
| op    | rs    | rt    | Indirizzo (offset) |

Esempio:  $1w \text{ } \$s2, \text{ } 8 (\$s3)$

L'indirizzo di memoria calcolato :

Contenuto di \$s3:      1100 1001 0001 0001 1011 1011 1010 1010 +  
offset (8):                          0000 0000 0000 1000 =

**Indirizzo:**                    1100 1001 0001 0001 1011 1011 1011 0010

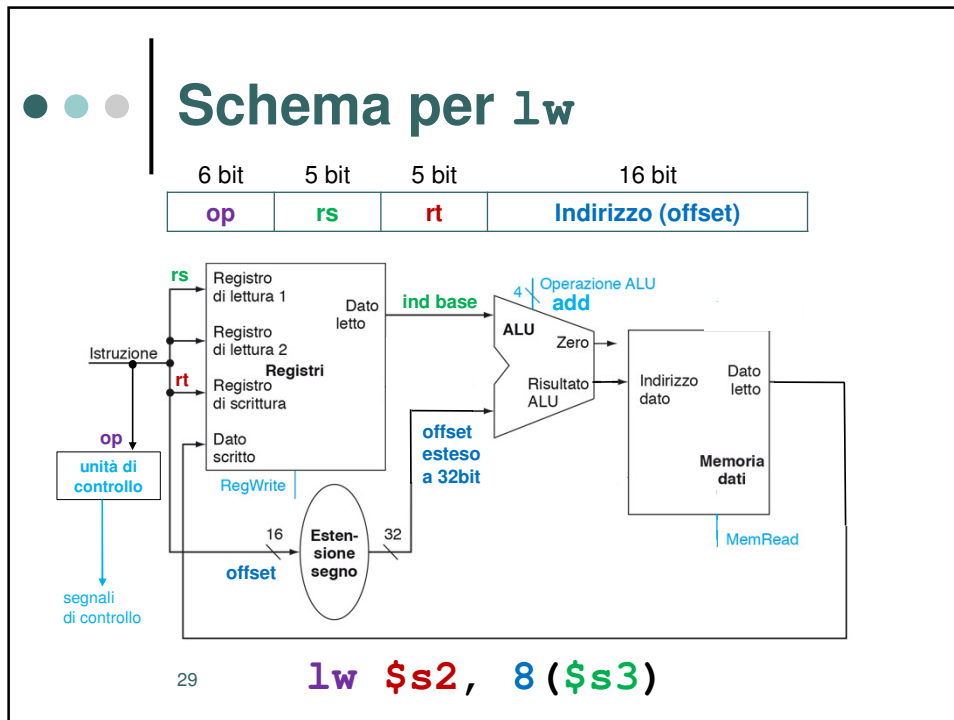
27



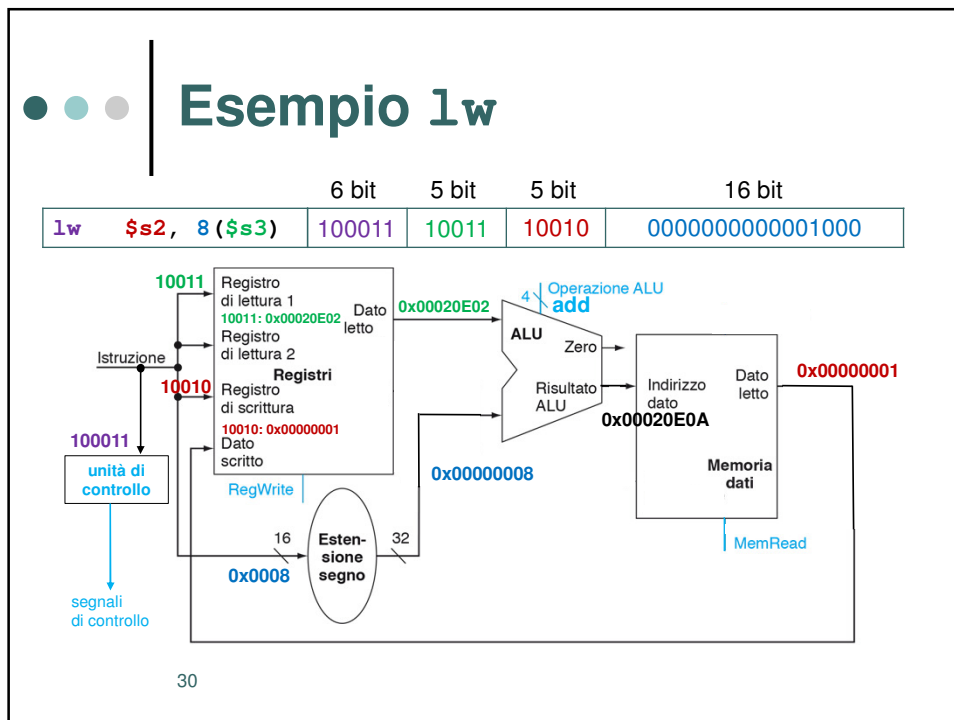
## Sequenza azioni $1w$

1. Lettura del registro base dal banco dei registri (**lettura Registri**)
2. Calcolo dell'indirizzo di memoria: registro base + offset (**ALU**)
3. Lettura del dato dalla memoria all'indirizzo calcolato (**lettura Memoria**)
4. Scrittura del dato nel registro destinazione (**scrittura Registri**)

28



29



30



## Sequenza azioni sw

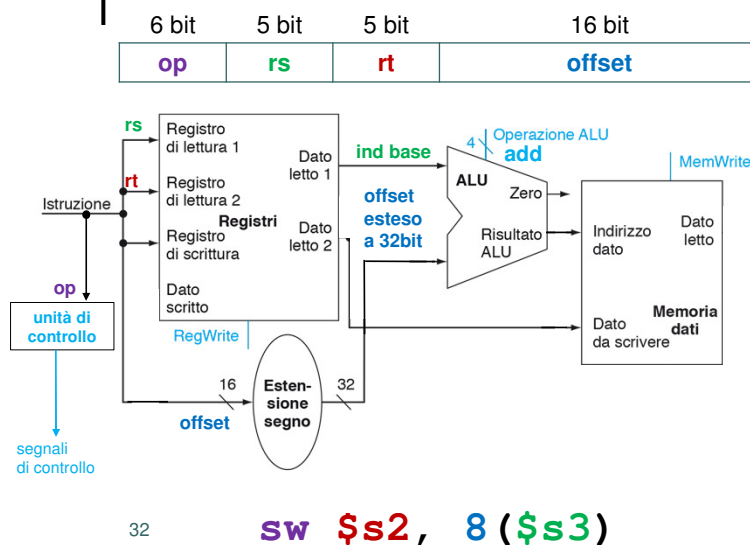
1. Lettura del registro base e del registro sorgente dal banco dei registri (**lettura Registri**)
2. Calcolo dell'indirizzo di memoria: registro base + offset (**ALU**)
3. Scrittura, del dato contenuto nel registro sorgente, nella memoria all'indirizzo calcolato (**scrittura Memoria**)

31

31

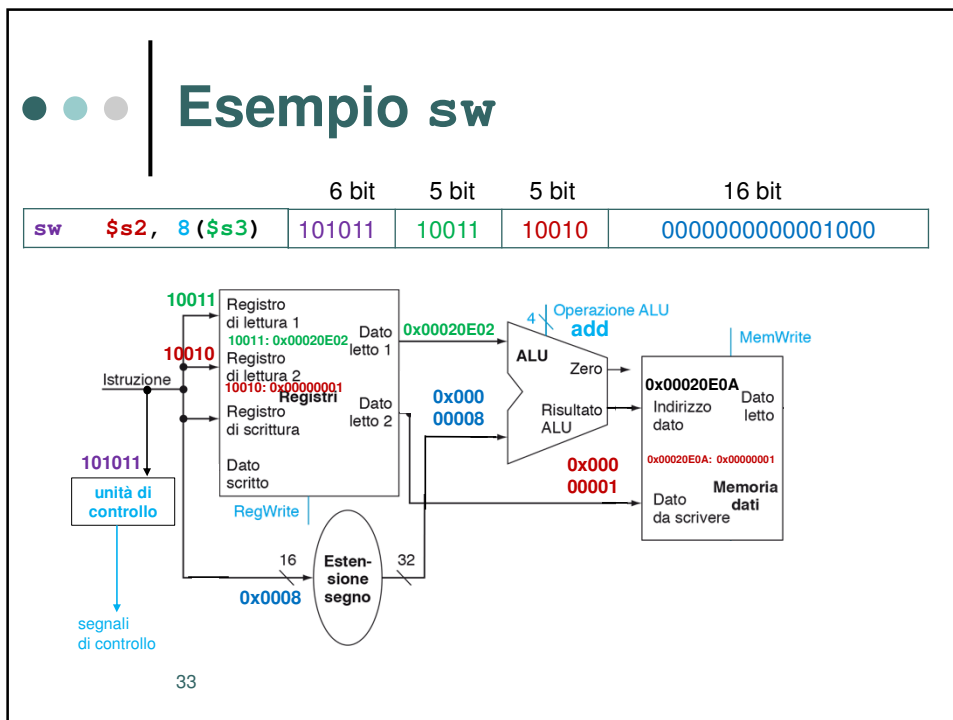


## Schema per sw

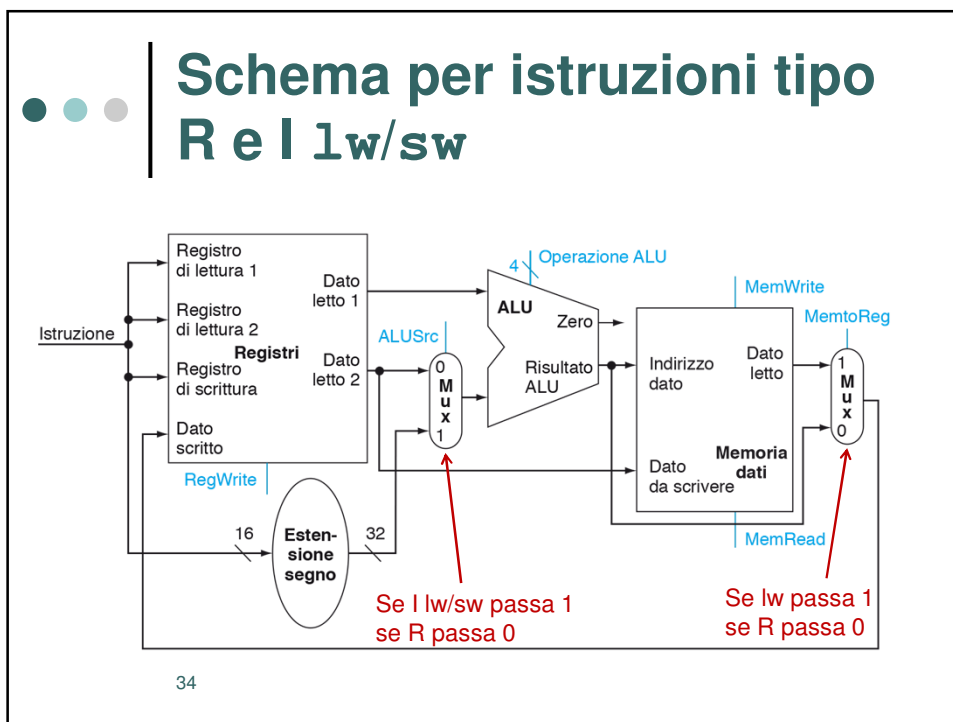


32





33



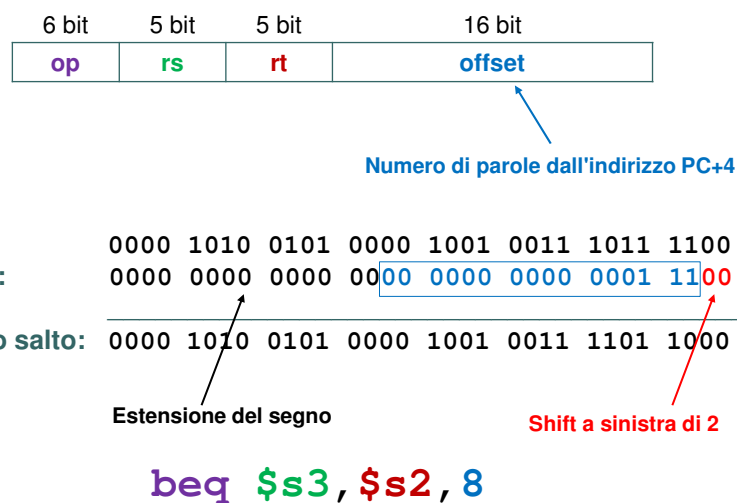
34

## Esecuzione delle istruzioni di formato I (salto condizionato)

35

35

### Istruzione di salto condizionato: beq



36



## Sequenza azioni beq

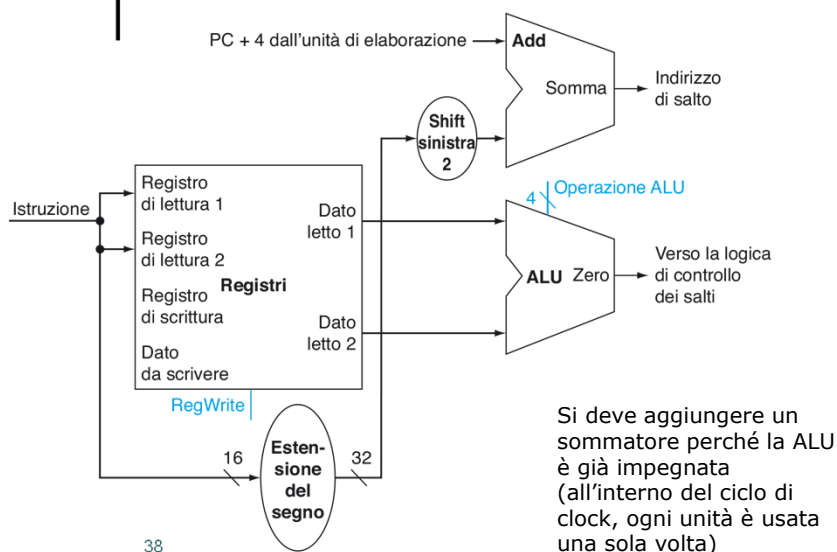
1. Calcolo dell'indirizzo di salto:
  1. Estensione dell'offset su 32 bit
  2. Moltiplicazione per 4 dell'offset
  3. Somma del PC con l'estensione del segno.
2. Controllo se il contenuto dei registri è uguale
  1. se uguale, l'indirizzo della prossima istruzione sarà quello calcolato
  2. se diverso sarà l'indirizzo in sequenza

37

37



## Calcolo indirizzo di salto

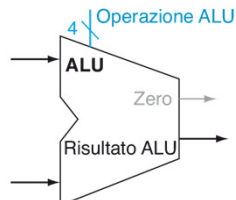


38

38



## Confronto tra registri



```
beq $s3, $s2, L1
```

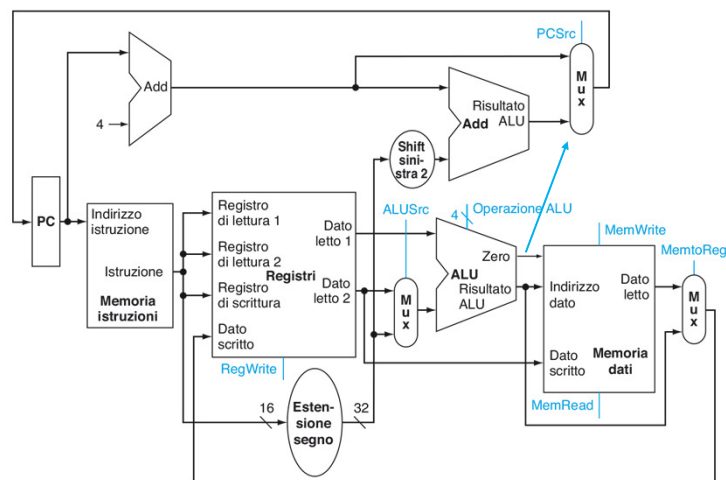
1. la ALU effettua la **sottrazione** fra i **2 registri**
2. se i contenuti sono **uguali**, in uscita della ALU si ha il bit **Zero=1**
3. logica di controllo:
  1. Se Zero = 1 allora  
PC = l'indirizzo calcolato a partire dall'**offset**
  2. Se Zero = 0 allora  
PC = PC+4

39

39



## Schema per R e I



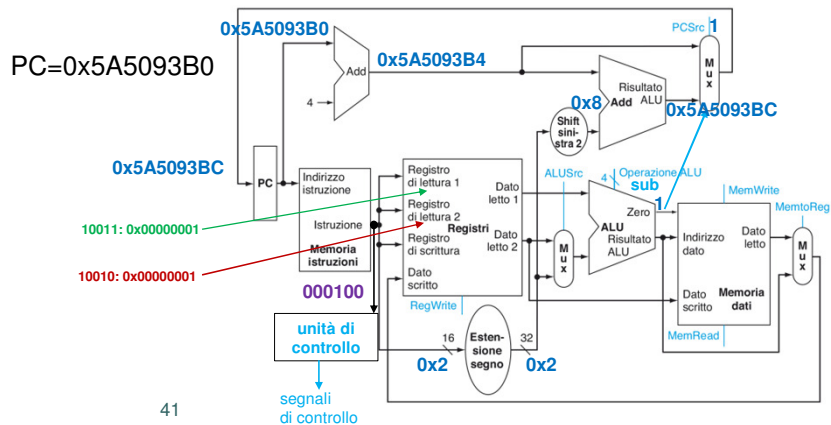
40

40



## Esempio beq

|                    |        |       |       |                  |
|--------------------|--------|-------|-------|------------------|
| beq \$s3, \$s2, L1 | 000100 | 10011 | 10010 | 0000000000000010 |
|--------------------|--------|-------|-------|------------------|



41

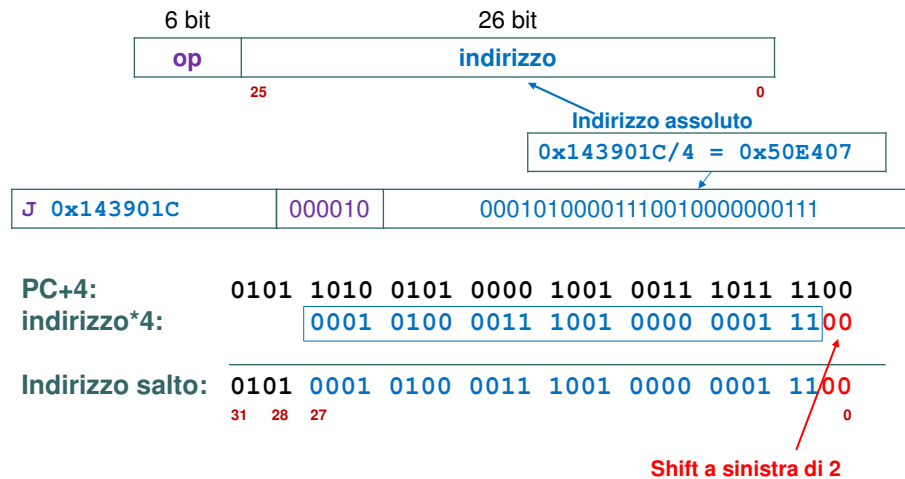


## Esecuzione della istruzione di formato J (salto incondizionato)

42

42

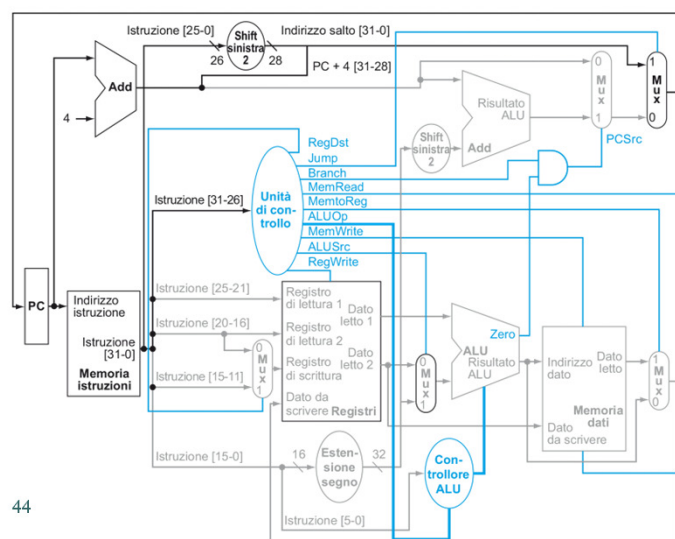
## Istruzione di salto in condizionato: j



43

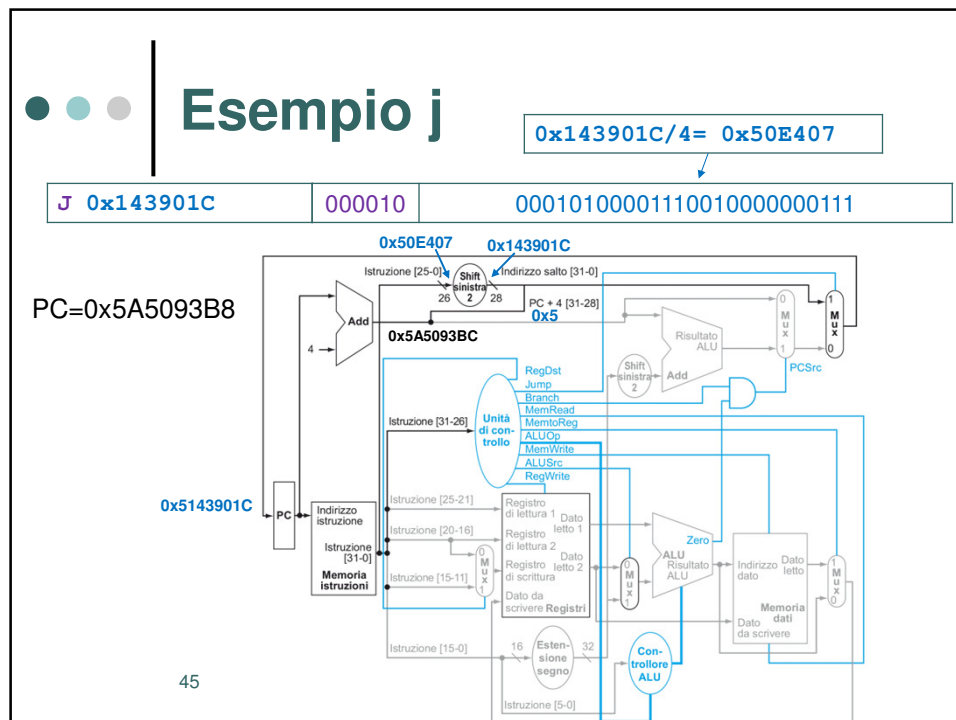
43

## Schema complessivo



44

44



45

**Riassunto - CPU ciclo unico**

1. L'esecuzione di una istruzione si compie in **un ciclo** di clock
2. Tutte le unità coinvolte nell'esecuzione di un'istruzione sono utilizzate **contemporaneamente**
3. Ogni unità può essere usata **una sola volta** (per ogni istruzione)

Questo implica che:

1. È necessario avere **2 memorie** (istruzioni e dati)
2. Abbiamo **3 ALU** (1 ALU e 2 sommatori)
3. Dobbiamo usare alcuni **multiplexer**

46

46



## Esercizi da fare a casa 1

Si assuma che

- o \$s1 contenga 0x4
- o \$s2 contenga 4
- o \$s3 contenga 0x2A
- o \$s4 contenga 0x30000018
- o PC contenga 0x53440040 (indirizzo istruzione)
- o L1 sia 2
- o parte di memoria dati

|            |            |
|------------|------------|
| 0x30000010 | 0x00000000 |
| 0x30000014 | 0x00000000 |
| 0x30000018 | 0x00000001 |
| 0x3000001C | 0x00000002 |
| 0x3000001E | 0x00000003 |

Descrivere l'esecuzione delle seguenti istruzioni (usare lo schema per R e I)

1. `add $s5, $s1, $s2`
2. `sub $s5, $s1, $s2`
3. `and $s5, $s1, $s2`
4. `or $s5, $s1, $s2`
5. `slt $s5, $s1, $s2`
6. `lw $s1, 4($s4)`
7. `sw $s5, 4($s4)`
8. `beq $s1, $s2, L1`
9. `beq $s1, $s3, L1`

Descrivere l'esecuzione della seguente istruzione (usare lo schema per J)

10. `j 32`

47

47



## Esercizi da fare a casa 2

Risolvere **l'esempio di esame seconda parte** in 90 minuti.

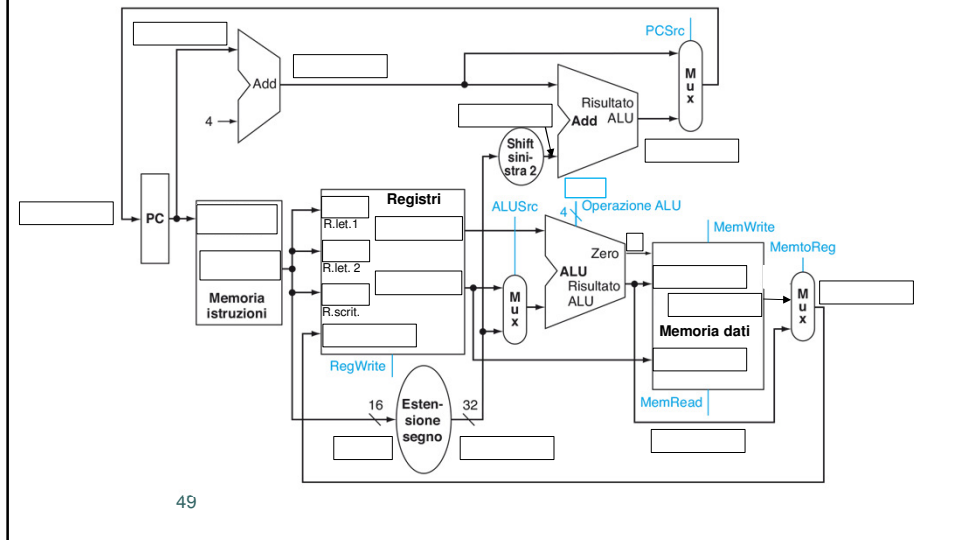
48

48





## Schema per gli esercizi R e I

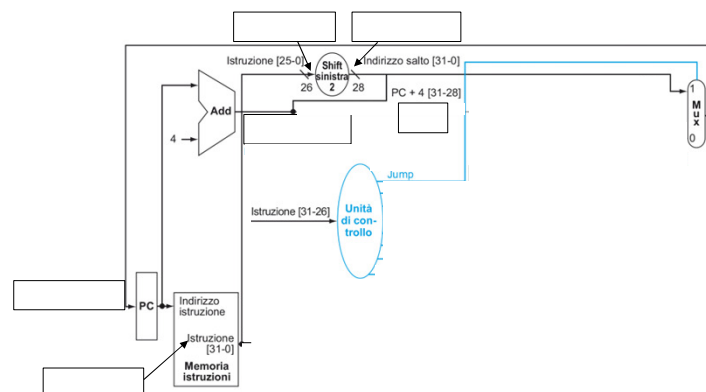


49

49



## Schema per gli esercizi J



50

50



# Architettura degli elaboratori

## Lezione 9 - esercizi

*Prof.ssa Valentina Ciriani*

Università degli Studi di Milano

[www.di.unimi.it/ciriani](http://www.di.unimi.it/ciriani)

51



## Esercizio 1

Si supponga che `$t0` contenga il valore `0x00101000`.  
Determinare il contenuto di `$t2` dopo l'esecuzione delle seguenti istruzioni:

```
$t0: 0000 0000 0001 0000 0001 0000 0000 0000
      slt $t2, $zero, $t0 # 0<1.052.672
$t2: 0000 0000 0000 0000 0000 0000 0000 0001
      bne $t2, $zero, ELSE # 1!=0 va a ELSE
      j FINE
ELSE: addi $t2, $t2, 2      # 1+2=3
$t2: 0000 0000 0000 0000 0000 0000 0000 0011
FINE:
$t2: 0000 0000 0000 0000 0000 0000 0000 0011
```

52

52



## Esercizio 2.a

Si supponga che il prossimo valore del program counter (PC+4) sia  $0x20000000$ . Qual è l'insieme degli indirizzi che il PC può assumere dopo che è stata eseguita l'istruzione di jump incondizionata (j)?

Posso utilizzare 26 bit per la jump

```
PC+4: 0010 0000 0000 0000 0000 0000 0000 0000
min:  0010 0000 0000 0000 0000 0000 0000 0000
max:  0010 1111 1111 1111 1111 1111 1111 1100
```

$[0x20000000, 0x2FFFFFFC]$

53

53



## Esercizio 2.b

Si supponga che il prossimo valore del program counter (PC+4) sia  $0x20000000$ . Quale intervallo di indirizzi si può raggiungere utilizzando l'istruzione MIPS beq?

Istruzione contenente 16 bit per indirizzo relativo con valori in  $[-2^{15}, 2^{15}-1] = [-32768, 32767]$

```
PC+4: 0010 0000 0000 0000 0000 0000 0000 0000
      536.870.91210
min:  0001 1111 1111 1110 0000 0000 0000 0000
      (536870912 - 32768×4)10
max:  0010 0000 0000 0001 1111 1111 1111 1100
      (536870912 + 32767×4)10
```

$[0x1FFE0000, 0x2001FFFC]$

54

54



## Esercizio 3

Tradurre le seguenti istruzioni in linguaggio macchina

|      |                  |        |       |       |                  |       |        |
|------|------------------|--------|-------|-------|------------------|-------|--------|
| addi | \$t0, \$s6, 4    | 001000 | 10110 | 01000 | 0000000000000100 |       |        |
| add  | \$t1, \$s6, \$s0 | 000000 | 10110 | 10000 | 01001            | 00000 | 100000 |
| sw   | \$t1, 0(\$t0)    | 101011 | 01000 | 01001 | 0000000000000000 |       |        |
| lw   | \$t0, 0(\$t0)    | 100011 | 01000 | 01000 | 0000000000000000 |       |        |
| add  | \$s0, \$t1, \$t0 | 000000 | 01001 | 01000 | 10000            | 00000 | 100000 |

```

00100010110010000000000000000100
00000010110100000100100000100000
10101101000010010000000000000000
10001101000010000000000000000000
000000010010100010000000000100000

```

55

55



## Esercizio 4

Tradurre in notazione esadecimale la seguente istruzione assembler:

sw \$t1, 32(\$t2)

e determinare il suo formato

|                   |        |       |       |                |  |  |  |
|-------------------|--------|-------|-------|----------------|--|--|--|
| sw \$t1, 32(\$t2) | 101011 | 01010 | 01001 | 00000000010000 |  |  |  |
|-------------------|--------|-------|-------|----------------|--|--|--|

1010 1101 0100 1001 0000 0000 0010 0000

0xAD490020

Il formato è I

56

56



## Esercizio 5

Scrivere l'istruzione assembler associata ai seguenti campi del MIPS: op = 0, rs = 8, rt = 9, rd = 8, shamt = 0, funz = 34 individuare il formato e scrivere la rappresentazione binaria

|                      |        |       |       |       |       |        |
|----------------------|--------|-------|-------|-------|-------|--------|
| sub \$t0, \$t0, \$t1 | 000000 | 01000 | 01001 | 01000 | 00000 | 100010 |
|----------------------|--------|-------|-------|-------|-------|--------|

0000 0001 0000 1001 0100 0000 0010 0010  
0x01094022 Formato R

57

57



## Esercizio 6

Scrivere l'istruzione assembler associata ai seguenti campi del MIPS: op = 0x23, rs = 9, rt = 12, costante = 0x4 individuare il formato e scrivere la rappresentazione binaria

0x23 = 0010 0011

|                  |        |       |       |                  |
|------------------|--------|-------|-------|------------------|
| lw \$t4, 4(\$t1) | 100011 | 01001 | 01100 | 0000000000000100 |
|------------------|--------|-------|-------|------------------|

1000 1101 0010 1100 0000 0000 0000 0100  
0x8D2C0004  
Formato I

58

58



## Esercizio 7

Si consideri una nuova istruzione chiamata `rpt`. Questa istruzione combina in un'unica istruzione la verifica di una condizione di fine ciclo con il decremento dell'indice del ciclo.

Per esempio,

```
rpt $s0, ciclo
```

corrisponde al seguente frammento di codice (con `x` in `$s0`):

```
if (x > 0) {  
    x = x - 1  
    goto ciclo}
```

- a) Quale sarebbe il formato di istruzione più appropriato per implementare l'istruzione (`R`, `I` o `J`)? **Formato I**

Determinare una sequenza di istruzioni che consenta di implementare l'istruzione riportata sopra

```
ciclo:  
    slt $t0, $zero, $s0      # se x>0 $t0=1  
    beq $t0, $zero, fuori    # se x<=0 vai a fuori  
    addi $s0,$s0,-1          # x=x-1  
    j  ciclo                 # vai a ciclo  
fuori:
```

59

59



## Esercizio 8.a

Si consideri il seguente ciclo in assembler MIPS:

```
CICLO: slt  $t2, $zero, $t1  
        beq  $t2, $zero, FINE  
        addi $t1, $t1, -1  
        addi $s2, $s2, 2  
        j    CICLO  
FINE:
```

Supponendo che `$t1` sia

inizializzato a 10 e `$s2` a 0. Quale è il valore finale di `$s2`?

| ciclo | \$t1 | \$t2 | \$s2 |
|-------|------|------|------|
| 0     | 10   | -    | 0    |
| 1     | 9    | 1    | 2    |
| 2     | 8    | 1    | 4    |
| 3     | 7    | 1    | 6    |
| 4     | 6    | 1    | 8    |
| ....  |      |      |      |
| 9     | 1    | 1    | 18   |
| 10    | 0    | 1    | 20   |
| FINE  | 0    | 0    | 20   |

60

60



## Esercizio 8.b

Si traduca il seguente ciclo in un linguaggio ad alto livello. Si supponga che i registri `$s1`, `$s2`, `$t1` e `$t2` contengano rispettivamente le variabili intere `A`, `B`, `i` e `temp`.

Consideriamo `$t1` inizializzato a 10.

```
CICLO: slt  $t2, $zero, $t1
        beq  $t2, $zero, FINE
        addi $t1, $t1, -1
        addi $s2, $s2, 2
        j    CICLO
```

FINE:

linguaggio ad alto livello:

```
for (i=10; i>0; i--) {B+=2}
```

61

61



## Esercizio 9

Tradurre il seguente frammento di codice in codice assembler MIPS. Si supponga che le variabili `a`, `b`, `i` e `j` siano contenute rispettivamente nei registri `$s0`, `$s1`, `$t0` e `$t1`. Si supponga che il registro `$s2` contenga l'indirizzo base del vettore `D`

```
for i=0; i<a; i++
{for j=0; j<b; j++
 {D[4*j] = i+j}}
```

62

```
addi $t0, $zero, 0
Loop1: slt  $t2, $t0, $s0
        beq  $t2, $zero, Fine1
        addi $t1, $zero, 0
Loop2: slt  $t3, $t1, $s1
        beq  $t3, $zero, Fine2
        add  $t4, $t0, $t1
        sll  $t5, $t1, 4
        add  $t6, $t5, $s2
        sw   $t4, 0($t6)
        addi $t1, $t1, 1
        j    Loop2
Fine2: addi $t0, $t0, 1
        j    Loop1
Fine1:
```

62



## Esercizio 10

Tradurre il seguente ciclo in un linguaggio ad alto livello. Si supponga che il registro \$t1 contenga la variabile intera *i*, \$s2 contenga la variabile intera *Ris* e \$s0 l'indirizzo base del vettore MemVett.

```
CICLO:    addi $t1, $zero, 0
          lw  $s1, 0($s0)
          add $s2, $s2, $s1
          addi $s0, $s0, 4
          addi $t1, $t1, 1
          slti $t2, $t1, 100
          bne $t2, $zero, CICLO
```

```
for (i=0; i<100; i++)
    {Ris += MemVett[i]}
```

63